Program 1

Problem Statement
Implement an Employee Attendance System as a single Employee class. The class captures employee details, a fixed working window, and 30 days of entry/leave stamps (read from the system clock). It computes per-day work hours, weekly overtime, a 10% bonus, and deductions for late entries / early leaves, then prints the full attendance record and a final salary breakdown.

Source Code:

```
#include <bits/stdc++.h>
using namespace std;

class Employee
{
public:
string name;
int id;
double basic_salary;
double bonus;
int late_days;
int early_leave_days;
int entry_hour;
int entry_minute;
string entry_period;
int leave_hour;
int leave_minute;
string leave_period;
string date[50];
string entry_time[50];
string leave_time[50];

double daily_work_hours[50];

Employee()
{
late_days = 0;
early_leave_days = 0;
bonus = 0;
for (int i = 0; i < 30; i++)
{
daily_work_hours[i] = 0;
}
}
```

```
int convertToMinutes(int hour, int minute)
{
return hour * 60 + minute;
}

void getCurrentTime(int &hour, int &minute)
{
time_t now = time(0);
tm *localTime = localtime(&now);
hour = localTime->tm_hour;
minute = localTime->tm_min;
}

string formatTime(int hour, int minute)
{
string period;
int displayHour;
if (hour == 0)
displayHour = 12;
else if (hour > 12)
displayHour = hour - 12;
else
displayHour = hour;

string result;
result = (displayHour < 10 ? "0" : "")
+ to_string(displayHour) + ":"
+ (minute < 10 ? "0" : "")
+ to_string(minute) + " "
+ period;
return result;
}

void setWorkingTime()
{
cout << "Enter fixed entry time\n";
cout << "Hour (0-24): ";
cin >> entry_hour;
cout << "Minute (0-59): ";
cin >> entry_minute;
cout << "\nEnter fixed leave time\n";
cout << "Hour (0-24): ";
```

```
cin >> leave_hour;

cout << "Minute (0-59): ";
cin >> leave_minute;

cout << "\nWorking time: "
<< formatTime(entry_hour, entry_minute)
<< " to "
<< formatTime(leave_hour, leave_minute)
<< "\n";
}

void takeEntryRecords()
{
cout << "\nEnter attendance for 30 days\n";

for (int i = 0; i < 30; i++)
{
cout << "\nDay " << i + 1;
cout << "\nPress Enter to record entry time";
cin.ignore();
cin.get();
int hour;
int minute;
getCurrentTime(hour, minute);
date[i] = "26/08/2026";
entry_time[i] =
formatTime(hour, minute);
int actualTime =
convertToMinutes(hour, minute);

int fixedTime =
convertToMinutes(entry_hour, entry_minute);
cout << "Date: " << date[i];
cout << "\nEntry: " << entry_time[i];

if (actualTime ⩽ fixedTime)
{
cout << "\nStatus: On Time";
}
else
{
```

```
cout << "\nStatus: Late";
late_days++;
}
cout << "\n";
}
}

void takeLeaveRecords()
{
cout << "\nEnter leave attendance for 30 days\n";

for (int i = 0; i < 30; i++)
{
cout << "\nDay " << i + 1;
cout << "\nPress Enter to record leave time";
cin.ignore();
cin.get();
int hour;
int minute;
getCurrentTime(hour, minute);
leave_time[i] =
formatTime(hour, minute);
int actualTime =
convertToMinutes(hour, minute);

int fixedTime =
convertToMinutes(leave_hour, leave_minute);

cout << "Date: " << date[i];
cout << "\nLeave: " << leave_time[i];

if (entry_hour > leave_hour)
{
if (actualTime < entry_hour * 60)
{
actualTime += 24 * 60;
}
if (fixedTime < entry_hour * 60)
{
fixedTime += 24 * 60;
}
}
```

```
if (actualTime ⩾ fixedTime)
{
cout << "\nStatus: On Time";
}
else
{
cout << "\nStatus: Early Leave";
early_leave_days++;
}

int entryMinutes;
entryMinutes = convertToMinutes(
stoi(entry_time[i].substr(0, 2)),
stoi(entry_time[i].substr(3, 2))
);

cout << "\n";
}
}

void calculateDailyWorkHours()
{
for (int i = 0; i < 30; i++)
{
int entryHour;
int entryMinute;

int leaveHour;
int leaveMinute;

entryHour = stoi(entry_time[i].substr(0, 2));
entryMinute = stoi(entry_time[i].substr(3, 2));
leaveHour = stoi(leave_time[i].substr(0, 2));
leaveMinute = stoi(leave_time[i].substr(3, 2));

string entryPeriod =
entry_time[i].substr(6, 2);

string leavePeriod =
leave_time[i].substr(6, 2);
```

```
int entryMinutes =
convertToMinutes(
entryHour,
entryMinute);
int leaveMinutes =
convertToMinutes(
leaveHour,
leaveMinute);
if (leaveMinutes < entryMinutes)
{
leaveMinutes += 24 * 60;
}

daily_work_hours[i] =
(leaveMinutes - entryMinutes) / 60.0;
}
}

double calculateOvertime()
{
calculateDailyWorkHours();
double overtimeSalary = 0;
for (int week = 0; week < 4; week++)
{
double weeklyHours = 0;
for (int day = 0; day < 7; day++)
{
int index = week * 7 + day;
weeklyHours += daily_work_hours[index];
}
if (weeklyHours > 40)
{
double extraHours =
weeklyHours - 40;

overtimeSalary +=
extraHours * 100;
}
}

return overtimeSalary;
}
```

```
void calculateBonus()
{
bonus = basic_salary * 0.10;
}

double calculateDeduction()
{
int totalBadDays =
late_days + early_leave_days;
int deductionDays =
totalBadDays / 3;
double oneDaySalary =
basic_salary / 30;
return deductionDays * oneDaySalary;
}

double calculateFinalSalary()
{
calculateBonus();

double overtime =
calculateOvertime();

double deduction =
calculateDeduction();

return basic_salary
+ bonus
+ overtime
- deduction;
}

void displayAttendance()
{
cout << "\nAttendance Record\n";

cout << left
<< setw(15) << "Date"
<< setw(15) << "Entry"
<< setw(15) << "Leave"
<< "\n";
```

```
for (int i = 0; i < 30; i++)
{
cout << left
<< setw(15) << date[i]
<< setw(15) << entry_time[i]
<< setw(15) << leave_time[i]
<< "\n";
}
}

void displaySalary()
{
calculateBonus();
double overtime =
calculateOvertime();
double deduction =
calculateDeduction();
double finalSalary =
calculateFinalSalary();

cout << "\nSalary Details\n";
cout << fixed << setprecision(2);
cout << "Employee ID: "
<< id << "\n";

cout << "Employee Name: "
<< name << "\n";
cout << "Basic Salary: "
<< basic_salary << "\n";
cout << "Bonus: "
<< bonus << "\n";

cout << "Late Entries: "
<< late_days << "\n";
cout << "Early Leaves: "
<< early_leave_days << "\n";
cout << "Overtime Salary: "
<< overtime << "\n";
cout << "Salary Deduction: "
<< deduction << "\n";
cout << "Final Salary: "
```

```
<< finalSalary << "\n";
}
};

int main()
{
Employee emp;

cout << "Employee Attendance System\n";
cout << "\nEnter Employee ID: ";
cin >> emp.id;
cin.ignore();
cout << "Enter Employee Name: ";
getline(cin, emp.name);
cout << "Enter Basic Salary: ";
cin >> emp.basic_salary;

cout << "Enter Basic Salary: ";
cin >> emp.basic_salary;
cout << "\n";

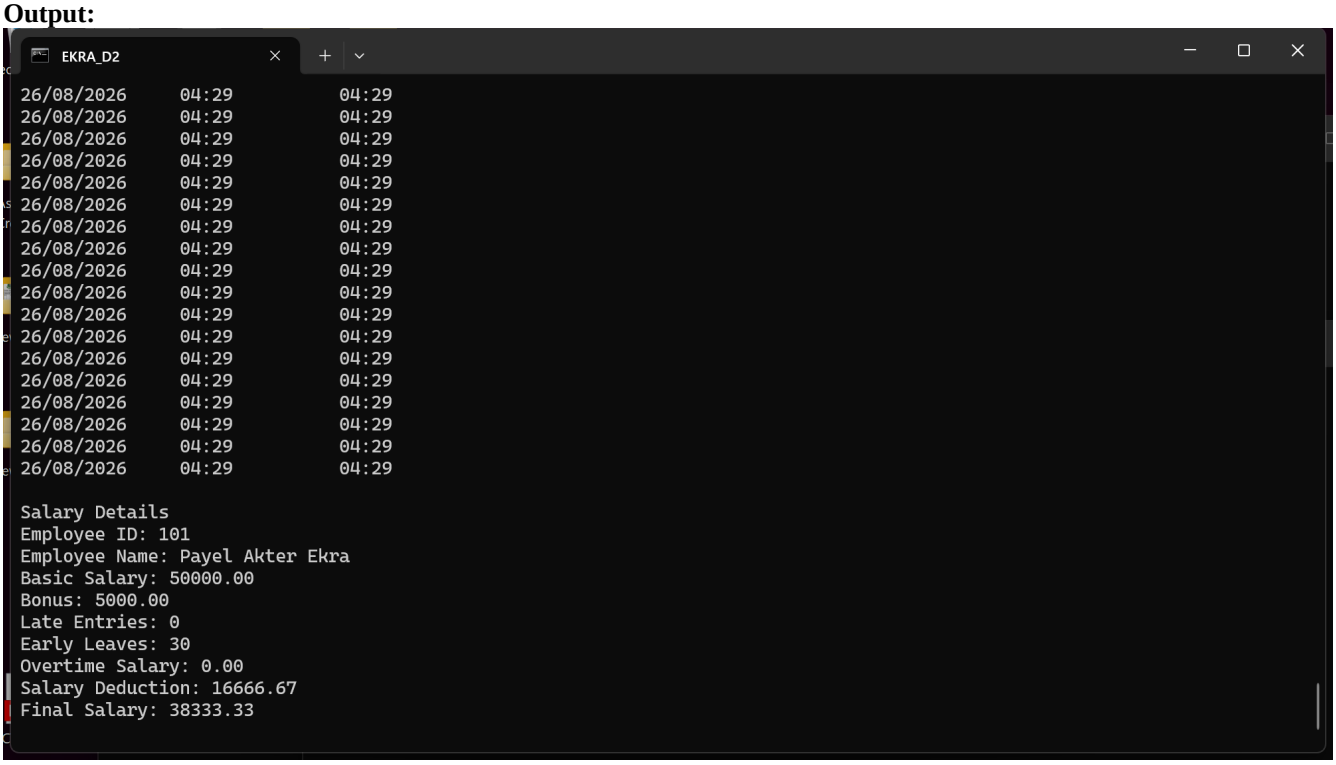
emp.setWorkingTime();
emp.takeEntryRecords();
emp.takeLeaveRecords();
emp.displayAttendance();
emp.displaySalary();

return 0;
}
```

Input:

```
101
Payel Akter Ekra
50000
50000
9
30
17
0
```

Output:


EKRA_D2
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
26/08/2026 04:29 04:29
Salary Details
Employee ID: 101
Employee Name: Payel Akter Ekra
Basic Salary: 50000.00
Bonus: 5000.00
Late Entries: 0
Early Leaves: 30
Overtime Salary: 0.00
Salary Deduction: 16666.67
Final Salary: 38333.33